

OWNER REFERENCE

How Star Player of the Week is built

A living document — updated as the app changes.

A plain reference for talking about this app with full confidence — what it's built with, how the data is organized, what's protected against, and how a change actually goes live.

01 WHAT IT'S BUILT WITH

FRONTEND

React + TypeScript

Built with Vite. Styled with Tailwind CSS on a small custom set of colors, spacing, and type sizes.

BACKEND

Firebase

No custom server, no Cloud Functions, and no sign-in of any kind — not even an invisible one. Firebase just stores the data.

DATABASE

Cloud Firestore

Data is stored as documents in named groups called "collections," not tables. The app gets live updates automatically, without asking for them.

HOSTING

Firebase Hosting

The site is hosted on Google's servers and deployed from the command line. There's no separate server to maintain.

UI PARTS

Radix UI + custom

Radix supplies accessible dialogs and toasts. Everything else — buttons, fields, badges — is built by hand to match the brand.

TESTING

Vitest + a Firestore test copy

Unit tests check plain logic. A separate test suite runs the real security rules against a local test copy of the database, to prove access rules actually work.

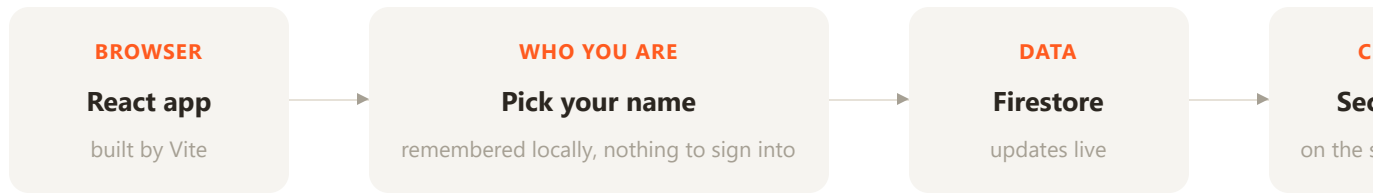
CODE STORAGE

GitHub

One main branch. Every change is saved as a commit before it

goes live.

02 HOW A REQUEST FLOWS THROUGH IT



Nobody signs in — not with a password, not even invisibly. Tapping a name on the roster just writes a note to your own browser's storage saying that's who you are; there's no account behind it and no server checking that the tap belongs to the right person. That was a deliberate trade: once the app stopped holding anything worth protecting that way, keeping any form of sign-in around stopped paying for itself.

There's no backend server deciding permissions — the app talks to the database directly, and a separate file called **Security Rules** decides what a given request is actually allowed to do. That matters less than it used to: with no identity behind any request, these rules mostly check the SHAPE of a write (is this really just a name? did the count only move by one?) rather than WHO is making it. The one thing they still gate on timing rather than identity is the vote count — see "What's protected" below.

Three names on the roster are still different: whoever's browser has claimed **KG**, **Steph**, or **OB** sees the session controls and the Team panel — see `REVEALER_NAMES` and `isRevealerProfile` in `types/firestore.ts`. Not a priority fallback: all three matter independently, and whichever of them happen to be claimed at once can each act. Voting starts every week LOCKED — `rollWeek` leaves `votingOpen: false` — and everyone signed in, revealer included, sees a "voting hasn't started" screen (see `VoteGrid`'s own empty state) until one of the three calls `startVoting`. From there, clicking Reveal Winner closes voting and works out the winner in the same action (see `doReveal` in `voting.service.ts`), so it's on whoever clicks it to have actually checked everyone's voted, by asking around the office. Unlike the single-host design this replaced, a revealer votes exactly like anyone else — declares stats, appears in the grid, casts a pick — this only ever gates the reveal action and the Team panel, never candidacy or the ability to vote. Separately, and regardless of who can currently reveal, **KG** specifically is never a candidate at all — never asked to declare stats, never on anyone's grid — because KG is the CEO, not because of anything to do with revealing. There's no role to assign and no account to promote; the app's own screen decides all of this purely by checking which names are currently claimed, the same as everything else about "who you are" here. Whoever it applies to sees a plain-language banner on their own screen saying so — see the `canReveal` block in `MainScreen.tsx` — so a Steph or OB isn't left guessing why they have extra controls. The name picker's own roster list only stars **KG** and **Steph** — deliberately not OB, so nobody glancing at that public list can tell OB can reveal too. Purely cosmetic: OB has the full power once claimed either way, just without the public tag — see `PUBLIC_REVEALER_NAMES` and `isPubliclyTaggedRevealer` in `types/firestore.ts`.

Reset Voting needs neither a reveal nor a close. Next to Reveal Winner, a revealer also has a **Reset Voting** button — clears this round's votes and locks voting again, any time, with no requirement that anything be revealed first. A fresh `startVoting` call is needed afterward, same as any other locked start — see the confirm copy in

`SessionControls.tsx`. It's the exact same underlying action ("Start a new vote") the results page already offered for after a reveal — see `startNewRound` in `voting.service.ts` — just made reachable mid-session too, for correcting a round before anyone's committed to revealing it. Nothing in the app ever flips `votingOpen` true except `startVoting` itself.

Reveal Winner only appears once a vote's been cast. With zero votes there's nothing to reveal, so the button simply isn't there — a short note sits in its place until the first vote lands (see `hasVotes` in `SessionControls.tsx`).

A tie is never broken — every tied name wins. There's no runoff and no re-vote: `computeWinners` returns everyone tied for the top count, however many that is, and they're all Star Players of the Week together. The reveal makes a moment of it — the wheel spins exactly as normal, then cracks down a jagged line and breaks apart, and every tied name pops in where it was (see `isTie / broken` in `Wheel.tsx`, the `.wheel-break-*` classes in `globals.css`, and `REVEAL_TIE_BREAK_MS`). Names are joined the way you'd say them — "A and B", "A, B and C" — via `joinNames` in `lib/winners.ts`, so a 6-way tie reads as naturally as a 2-way one.

A claim with nobody behind it frees itself after 15 minutes. Claiming a name has no expiry on its own — closing a tab doesn't release it — so every open, claimed browser quietly refreshes a `lastSeenAt` heartbeat on its own claim every 30s (see `touchClaim` in `claims.service.ts`), and every open tab — not just a revealer's — checks every 30s for any claim that's gone quiet longer than `CLAIM_STALE_AFTER_MS` and deletes it (`releaseStaleClaims`). This is what keeps the turnout count ("N people have voted so far") honest: without it, a name claimed once and then abandoned would count as signed in forever. `firestore.rules` only allows one kind of update to a claim — refreshing `lastSeenAt` alone — specifically so this can't become a way to repoint whose name it is.

Everyone but a revealer gets signed out after the reveal. Once the result is on screen, anyone who isn't one of KG/Steph/OB is automatically signed out a few seconds later — the same thing tapping Home does — and lands back on the name picker. A revealer stays, since the session controls (start a new round, manage the roster) are theirs to use. See `useAutoSignOutAfterReveal` in `ResultsPage.tsx`.

Home — one vote per device. The signed-in screens' Home button (which replaced "Not you?") signs the person out to the name picker. While voting's open it first takes back the vote this device cast (`retractVote`: -1 on the one count it added, plus deleting its own "I voted" marker), then frees the name and wipes its stats choice. Votes the person *received* sit on their own count and are never touched. A shared phone therefore can't stack votes; everyone votes on their own device.

Switching to Stats Down takes your votes away. You drop off the poll at once, and `doReveal` counts everyone's received votes except a Down person's (`countableTally` in `lib/roundRules.ts`, which also never counts KG), so they end on 0. Going Home or an expired sign-in never costs anyone the votes they'd received — and Home keeps a Down choice in place, so leaving can't undo Down. Received votes aren't deleted mid-round, just not counted, so no voter's device is ever left pointing at a count that's gone. Anyone who voted for a Down person sees "doesn't count any more" (`stalePick` in `MainScreen.tsx`) and can vote again, which takes their old -1 off as normal. If the person flips back to Up before the reveal, untouched votes count again.

Sign-in expiry can't sign out anyone who's really there. The 15-minute sweep (`findStaleClaimUids` in `lib/roundRules.ts`, unit-tested) never sweeps anyone Up this round, so a locked phone doesn't drop a candidate off the poll. A heartbeat still being written counts as fresh, and staleness is measured against the newest server heartbeat as well as the device clock, so one device with a wrong clock can't sign everyone out. It frees only the

claim, never the stats choice. If the server already holds a vote from you this round but your device has lost track of who for, you're told your vote is in instead of being allowed to vote twice (`votedWithoutPick`). Turnout's "N" is everyone signed in plus everyone who already voted (`roundParticipantCount`).

Everything resets after every reveal. `doReveal` saves the result (winner, total votes, in settings) and the streak history, and in that same write deletes every tally count, every voter marker and every stats choice, so each round starts from zero.

03 HOW THE DATA IS ORGANIZED

COLLECTION	WHAT IT HOLDS	WHO CAN READ IT
<code>sotw_profiles</code>	The roster — each teammate's name, plus their photo if they've added one. Nothing else	Everyone
<code>sotw_claims</code>	Which name has been tapped, plus a "last seen" heartbeat that browser refreshes every 30s. Who it's claimed by never changes after — only ever deleted and re-created	Everyone
<code>sotw_meta/settings</code>	Is voting open, which week it is, has it been revealed, who won	Everyone
<code>sotw_voters</code>	That a person has voted this week. There is no field here for who they voted for	Everyone
<code>sotw_tally</code>	The running count of votes for each candidate — numbers only, no names attached to them	Everyone, once voting closes
<code>sotw_stat_status</code>	Each person's own "stats up" or "stats down" for the current week	Everyone
<code>sotw_weekly_activity</code>	That someone received at least one vote in a given week — what the streak badges are counted from	Everyone

Every name starts with `sotw_` — a simple prefix so this data can share a database with something else later without clashing.

There is no collection for "who I voted for", and that absence is the design. Your own pick is written to your browser's storage and nowhere else, so the link between a person and the name they chose never reaches the database at all. The cost is small and deliberate: clear your browser data and the app forgets which card to tick for you — the vote itself still stands, you just can't change it from a browser that no longer remembers it.

04 WHAT'S PROTECTED

✓ **Nobody can learn who voted for whom.** It is never recorded — not for teammates, not for KG, Steph, or OB, not for anyone reading the raw database. Two things get stored: that a person voted, and a running count for each candidate, and neither field can be stretched to carry the other (the rules reject any extra field on either doc). This is a real guarantee, not a convention — it holds regardless of who's asking, because nobody is ever checked.

✓ **The count is unreadable while voting is open.** Whether it's a revealer, a teammate, or someone poking the database directly, the read is refused until voting closes — so nobody can watch a number tick up and work out who just voted. Once it closes, anyone can technically read it — but the app itself has no per-candidate view at all any more, for anyone: the only thing shown is turnout, and then the winner.

What's *not* a database-level guarantee any more, now that this app dropped every form of sign-in: "one name, one person" and "only KG, Steph, or OB can reveal" are conventions the app's own screens follow, not things the rules can verify — there's no identity left to check them against. A name being claimed by *somebody* can't be un-claimed out from under them by a second tap (the record is create-only), but nothing stops a browser from claiming a different name that isn't really theirs by going around the app's UI. For a small team with nothing valuable riding on it, that was judged an acceptable trade for having no sign-in step at all.

One further accepted trade-off, unrelated to identity: because the server holds no record of your pick, it can't check a vote being taken back against the vote you actually cast — it can only hold each change to a single point up or down. Someone hand-crafting requests could nudge a rival's count by one. Closing that would mean storing who voted for whom, which is precisely the thing this design refuses to do, so the one-point limit is where the line sits.

05 SHIPPING A CHANGE

Usually shipped together — the app and the rules it depends on:

```
# app code, and the rules it depends on, in one go
firebase deploy --only hosting,firestore:rules
```

They can still be deployed separately (`--only hosting` , or `--only firestore:rules` on their own) when that's genuinely useful — publishing a rules change ahead of the app code that needs it, say. But there's no longer a standing reason to keep them apart by default: with no accounts and no roles left to guard, a routine app update can't accidentally change who's allowed to do what, because nothing here is gated on who's asking any more.

06 WHAT'S HAPPENED SO FAR

Rebuilt from one big file

The app used to be a single large HTML file. It's now a proper React + TypeScript codebase, easier to maintain and grow. The old version is being phased out.

● **Security check-up**

A full review found and fixed real gaps — including a way anyone could inflate vote counts.

● **Made the session controls self-serve**

Adding, renaming and removing people, opening and closing voting, and recovering from a stuck state can now all be done from the app — no direct database edit needed.

● **Stats up or down, streaks, photos**

Everyone says whether their stats are up before they vote, and only "up" goes on the poll. Photos, a badge for anyone who's received votes three weeks running, and a "pause this week" switch for holidays landed alongside it.

● **Stripped back to just the vote**

Accounts and the Blacfox Dollars wallet are gone, and every screen that wasn't the vote went with them. Open the app, pick your name, vote — that's the whole app now.

● **Dropped the last trace of sign-in**

Even the invisible kind — picking your name used to hand your browser an anonymous Firebase identity behind the scenes, purely so the rules could enforce one name per browser. With nothing left worth protecting that way, that step is gone too, and "who you are" now just lives in your own browser's storage. The trade: one name per person is a convention the app follows now, not something the database can verify.

● **Picker became a dropdown**

The roster used to be a list of tap-to-claim cards. It's a select-and-join dropdown now, and a name drops out of the list entirely the moment someone takes it, rather than staying visible as a disabled row.